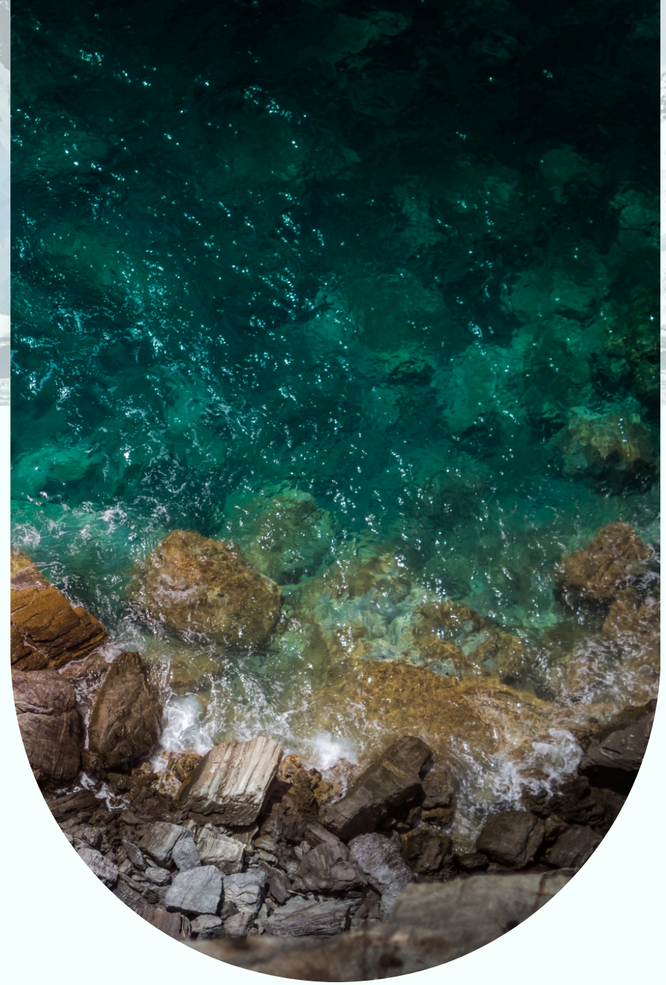


ANALYSE DES CATASTROPHES NATURELLES : CLUSTERING ET CLASSIFICATION DE LA SÉVÉRITÉ

Présenté par :
Sarra Askri

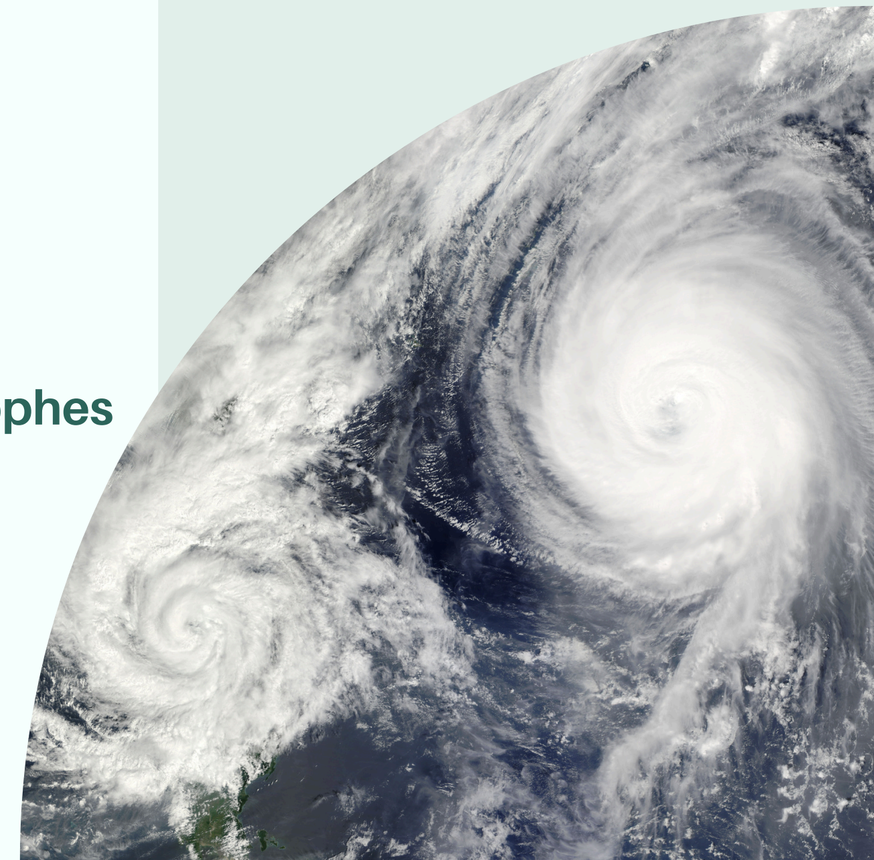
Année universitaire 2025/2026

Contexte

Les catastrophes naturelles ont des impacts importants sur les populations et les économies.
De nombreuses données sont disponibles pour décrire ces événements, mais leur complexité et leur diversité rendent leur exploitation difficile.

Comment exploiter les données pour prédire les catastrophes naturelles ?

- Analyser ces données.
- Utiliser des techniques pour regrouper les catastrophes similaires.
- Prédire leur niveau de sévérité.



Challenges :

La mise en œuvre de ce projet présente plusieurs défis importants :

- La qualité des données
- Le choix des variables pertinentes
- La construction d’un modèle de classification performant sans surapprentissage.
- La détermination des bons paramètres pour le clustering

Objectifs :

I- Préparation des données

Nettoyage des données, gestion des valeurs manquantes, suppression des doublons et traitement des valeurs aberrantes.

II- Analyse exploratoire des données

Étude de la distribution des variables et compréhension des caractéristiques principales des données.

III- Application du clustering

Utilisation de l'algorithme KMeans pour regrouper les catastrophes similaires et analyser les clusters obtenus.

IV- Mise en place de la classification

Construction et comparaison de plusieurs modèles de classification (Logistic Regression, Decision Tree, Random Forest, Gradient Boosting) en utilisant plusieurs splits des données.

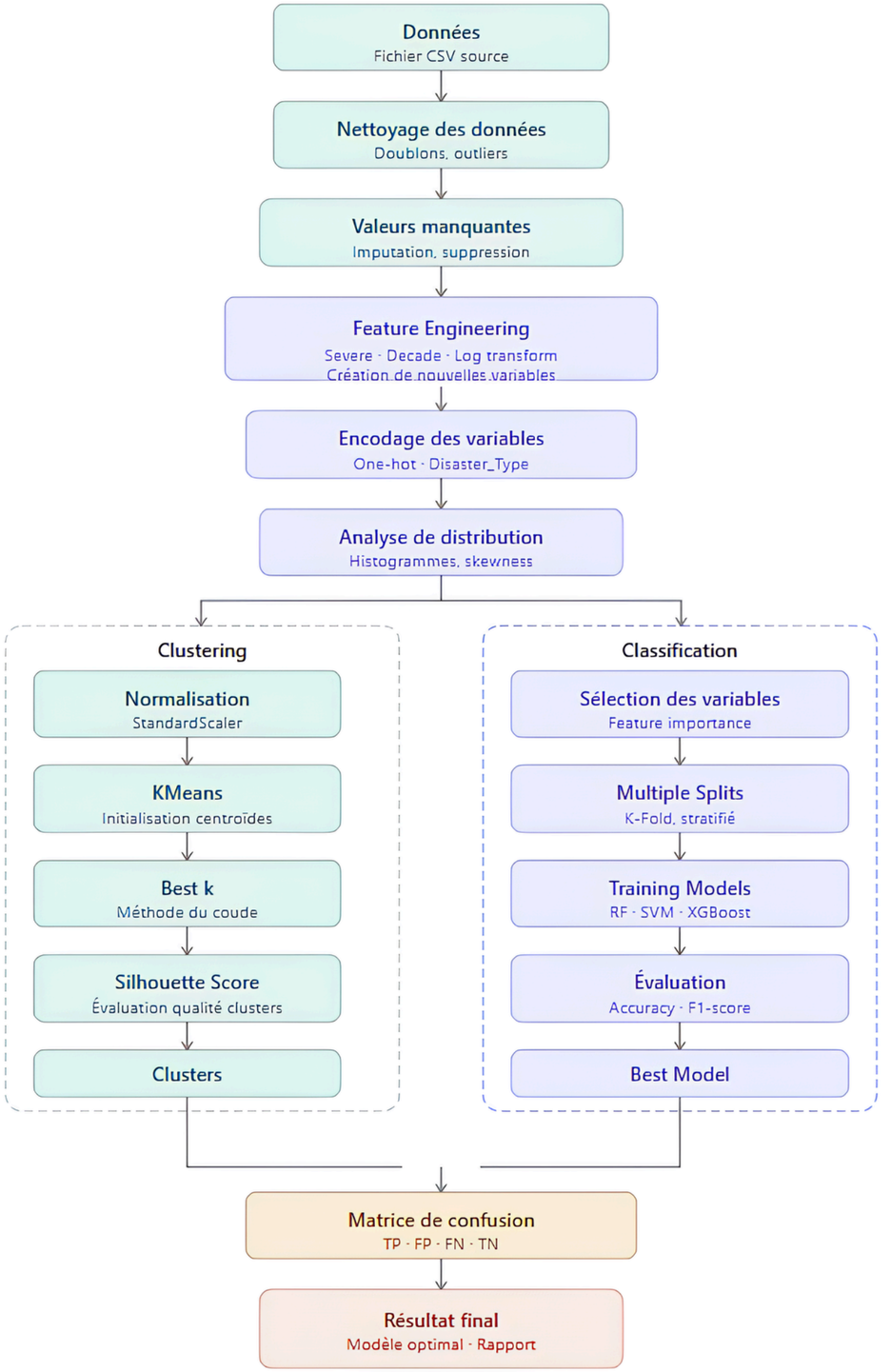
V- Évaluation des modèles

Analyse des performances à l'aide de métriques (Accuracy, F1-score) et sélection du meilleur modèle.

Flowchart :

Ce flowchart présente les étapes de notre approche, depuis la préparation des données jusqu'à la sélection du meilleur modèle, en passant par :

- Prétraitement et transformation des données
- Analyse exploratoire des variables
- Clustering avec KMeans (choix du meilleur k)
- Classification avec plusieurs modèles et plusieurs splits
- Évaluation des performances et sélection du meilleur modèle
- Analyse finale à l'aide de la matrice de confusion



I- Préparation des données

1 - Nettoyage des données

Cette étape consiste à supprimer les doublons et à corriger les types de données.

```
df = df.drop_duplicates()
# Convertir Year de manière sûre et retirer les lignes sans année exploitable.
df["Year"] = pd.to_numeric(df["Year"], errors="coerce")
rows_before = len(df)
df = df.dropna(subset=["Year"]).copy()
print("Lignes supprimées car l'année est manquante/invalide :", rows_before - len(df))
df["Year"] = df["Year"].astype(int)
print("Dimensions après nettoyage :", df.shape)
print("Type de Year :", df["Year"].dtype)
```

```
Lignes supprimées car l'année est manquante/invalide : 0
Dimensions après nettoyage : (11413, 45)
Type de Year : int64
```

2 - Gestion des valeurs manquantes

Cette étape consiste à remplacer ‘ Total_Deaths ’ par la médiane et ‘ Damage_USD’ par 0

.

```
df["Total_Deaths"] = df["Total_Deaths"].fillna(df["Total_Deaths"].median())
df["Damage_USD"] = df["Damage_USD"].fillna(0)

print(df[["Total_Deaths", "Damage_USD"]].isnull().sum())
```

✓ 0.0s

```
Total_Deaths    0
Damage_USD      0
dtype: int64
```

3 - Traitement des valeurs aberrantes

Dans cette étape, nous appliquons la méthode de capping basée sur l'IQR afin de traiter les valeurs extrêmes, tout en les conservant sans suppression.

```python
# Étape 7 : Traitement des outliers avec capping

Q1 = df["Total_Deaths"].quantile(0.25)
Q3 = df["Total_Deaths"].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
df["Total_Deaths_Capped"] = df["Total_Deaths"].clip(lower=lower_bound, upper=upper_bound)
print("Maximum avant capping :", df["Total_Deaths"].max())
print("Maximum après capping :", df["Total_Deaths_Capped"].max())
```

```
Maximum avant capping : 3700000.0
Maximum après capping : 148.5
```

→ Cette méthode permet de limiter les valeurs très élevées sans supprimer les lignes du dataset.
→ Cela permet d'éviter le problème d'une variable cible contenant une seule classe.

4 - Feature Engineering

Dans cette étape, nous créons la variable cible Severe, ainsi que la variable Decade, puis nous appliquons une transformation logarithmique sur la variable Damage_USD pour obtenir Damage_Log.

	Year	Decade	Total_Deaths	Total_Deaths_Capped	Damage_Thousands_USD	Damage_Log	Severe
0	1900	1900	11000.0	148.5	0.0	0.000000	1
1	1900	1900	1250000.0	148.5	0.0	0.000000	1
2	1902	1900	2000.0	148.5	25000.0	10.126671	1
3	1902	1900	1000.0	148.5	0.0	0.000000	1
4	1902	1900	6000.0	148.5	0.0	0.000000	1

→Decade : décennie de l'événement

→ Damage_Log : transformation logarithmique des dommages économiques exprimés en milliers de dollars, afin de réduire l'effet des grandes valeurs.

→ Severe : 1 si le nombre de décès est supérieur à 100, sinon 0.

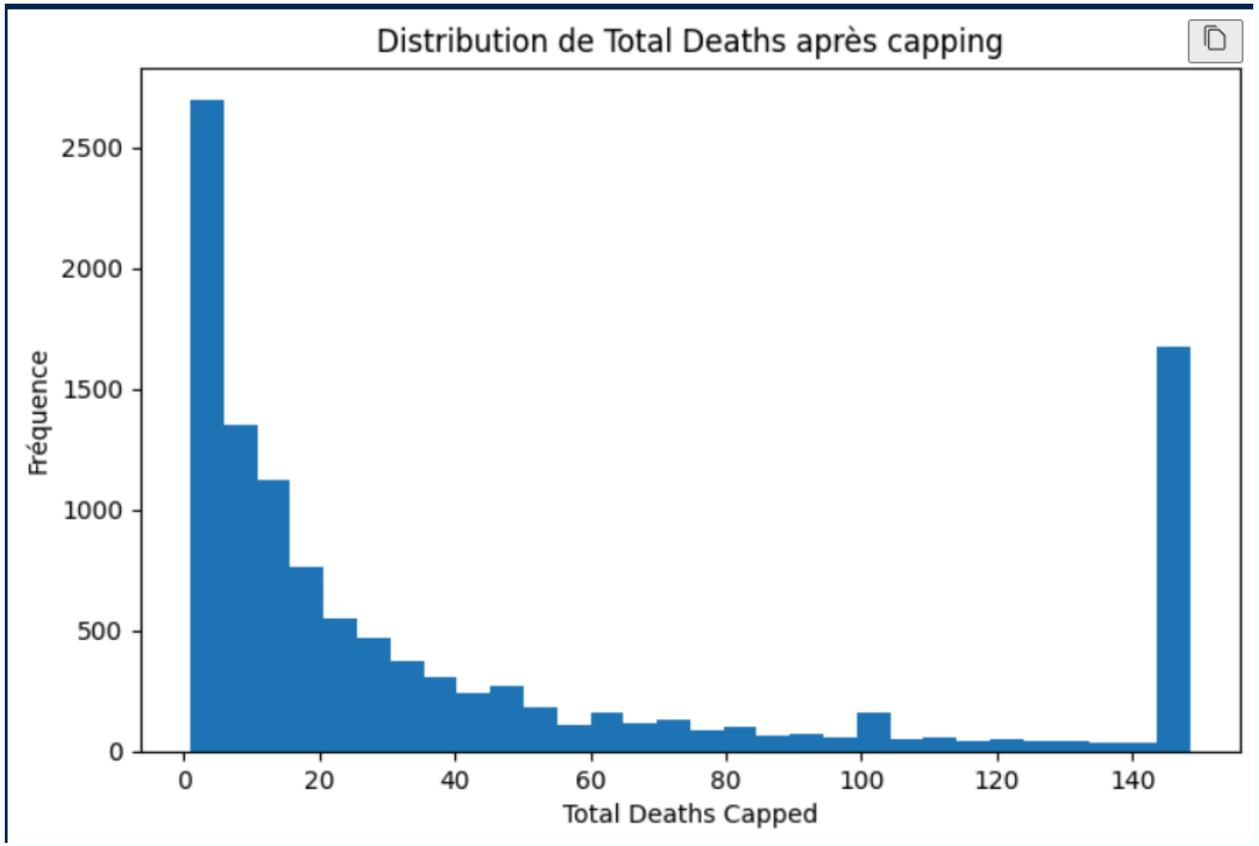
II- Analyse exploratoire des données

1. Analyse des distributions :

```
--- Analyse de distribution ---
       Total_Deaths  Total_Deaths_Capped  Damage_Thousands_USD     Damage_Log
count  1.141300e+04         11413.000000          1.141300e+04  11413.000000
mean   2.842866e+03            44.066854          3.016891e+05      3.909927
std    6.860595e+04            51.556452          3.207218e+06      5.477968
min    1.000000e+00             1.000000          0.000000e+00      0.000000
25%    6.000000e+00             6.000000          0.000000e+00      0.000000
50%    2.000000e+01            20.000000          0.000000e+00      0.000000
75%    6.300000e+01            63.000000          1.600000e+04      9.680406
max    3.700000e+06           148.500000          2.100000e+08     19.162618
```

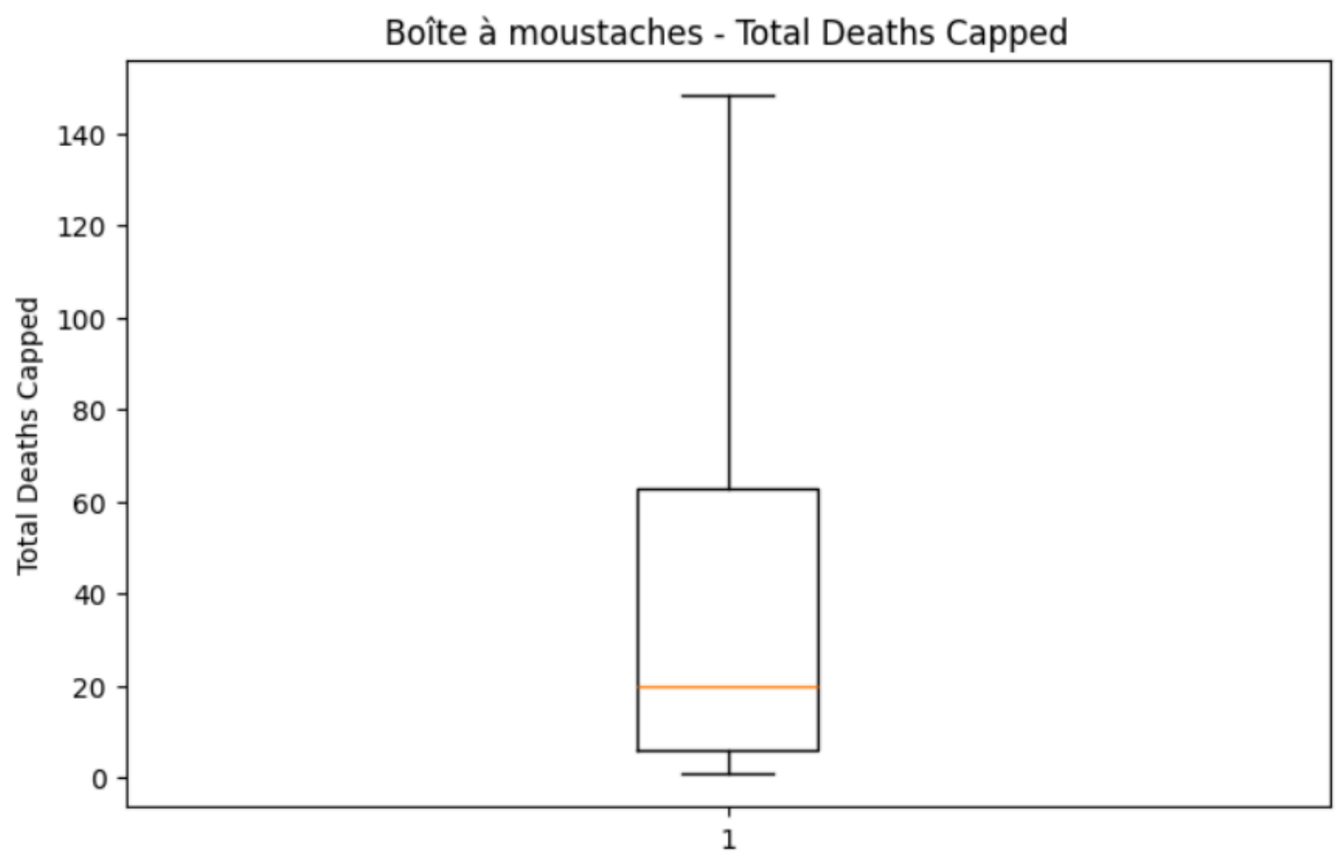
- L’analyse de distribution des variables permet d’identifier la dispersion des données ainsi que la présence de valeurs extrêmes.
- Elle aide également à comprendre la structure globale des données avant la modélisation.

→ Le principal bénéfice est d’améliorer la qualité des données et de guider les transformations nécessaires afin d’optimiser les performances des modèles.



2. Analyse des valeurs extrêmes :

Cette boîte à moustaches permet de visualiser la dispersion des données ainsi que les valeurs extrêmes.



→ **Cette étape permet de mieux comprendre la distribution des données et de préparer les transformations nécessaires.**

III- Application du clustering

Notre objectif est de regrouper les catastrophes similaires à l'aide de l'algorithme KMean et les données normalisées

1- Pour le clustering, nous utilisons les variables descriptives suivantes :

- Total_Deaths_Capped
- Damage_Log
- Year
- Decade
- Les variables du type de catastrophe

```
# Étape 11 : Préparation des variables pour le clustering
disaster_type_columns = [  col for col in df.columns if col.startswith("Disaster_Type_")]
clustering_features = [ "Total_Deaths_Capped", "Damage_Log", "Year"] + disaster_type_columns
X_clustering = df[clustering_features]
scaler_clustering = StandardScaler()
X_clustering_scaled = scaler_clustering.fit_transform(X_clustering)
print("Nombre de variables pour le clustering :", len(clustering_features))
```

```
Nombre de variables pour le clustering : 15
```

2- Choix du nombre de clusters :

Test de plusieurs valeurs de k utilisation :

- Méthode du coude (inertie)
- Silhouette Score

```
    silhouette_scores = []
    for k in k_values:
        kmeans_temp = KMeans(n_clusters=k, random_state=42, n_init=10)
        labels_temp = kmeans_temp.fit_predict(X_clustering_scaled)
        inertias.append(kmeans_temp.inertia_)
        silhouette_scores.append(silhouette_score(X_clustering_scaled, labels_temp))
    kmeans_results = pd.DataFrame({
        "k": list(k_values),
        "Inertie": inertias,
        "Silhouette Score": silhouette_scores
```

3. Recherche du meilleur nombre de clusters:

Pour évaluer la qualité du clustering, nous utilisons :

- valeurs de k comprises entre 2 et 8
- Silhouette Score : plus il est proche de 1, meilleure est la séparation entre les clusters.

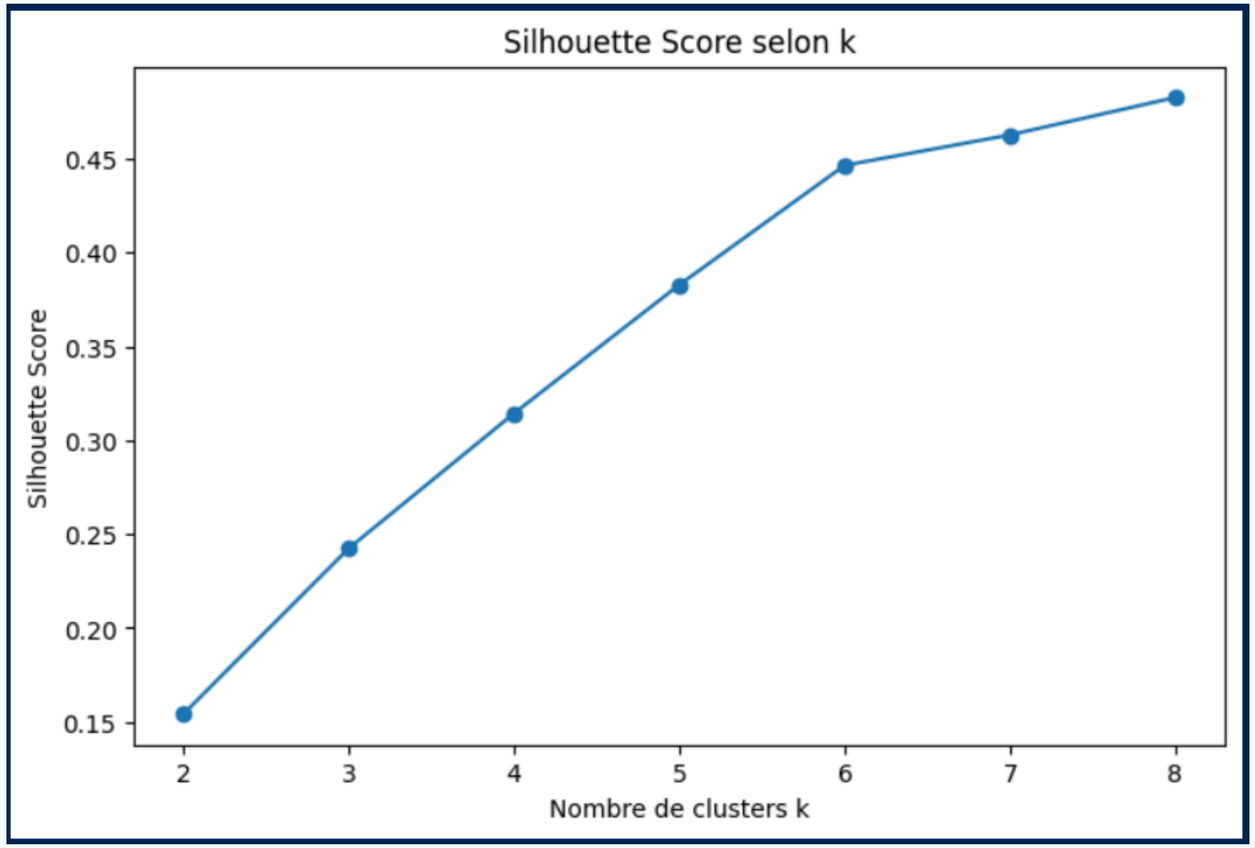
```
# Étape 12 : Recherche du meilleur k
k_values = range(2, 9)
inertias = []
silhouette_scores = []
for k in k_values:
    kmeans_temp = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels_temp = kmeans_temp.fit_predict(X_clustering_scaled)
    inertias.append(kmeans_temp.inertia_)
    silhouette_scores.append(silhouette_score(X_clustering_scaled, labels_temp))
kmeans_results = pd.DataFrame({
    "k": list(k_values),
    "Inertie": inertias,
    "Silhouette Score": silhouette_scores
})

kmeans_results
```

	k	Inertie	Silhouette Score
0	2	154656.383105	0.154306
1	3	139887.526314	0.242449
2	4	126422.816950	0.314325
3	5	113310.506292	0.382750
4	6	100438.143041	0.446322
5	7	88844.152333	0.462518
6	8	77224.893245	0.482470

4. Résultat :

Nous affichons le graphe :

- le score de silhouette en fonction du nombre de clusters



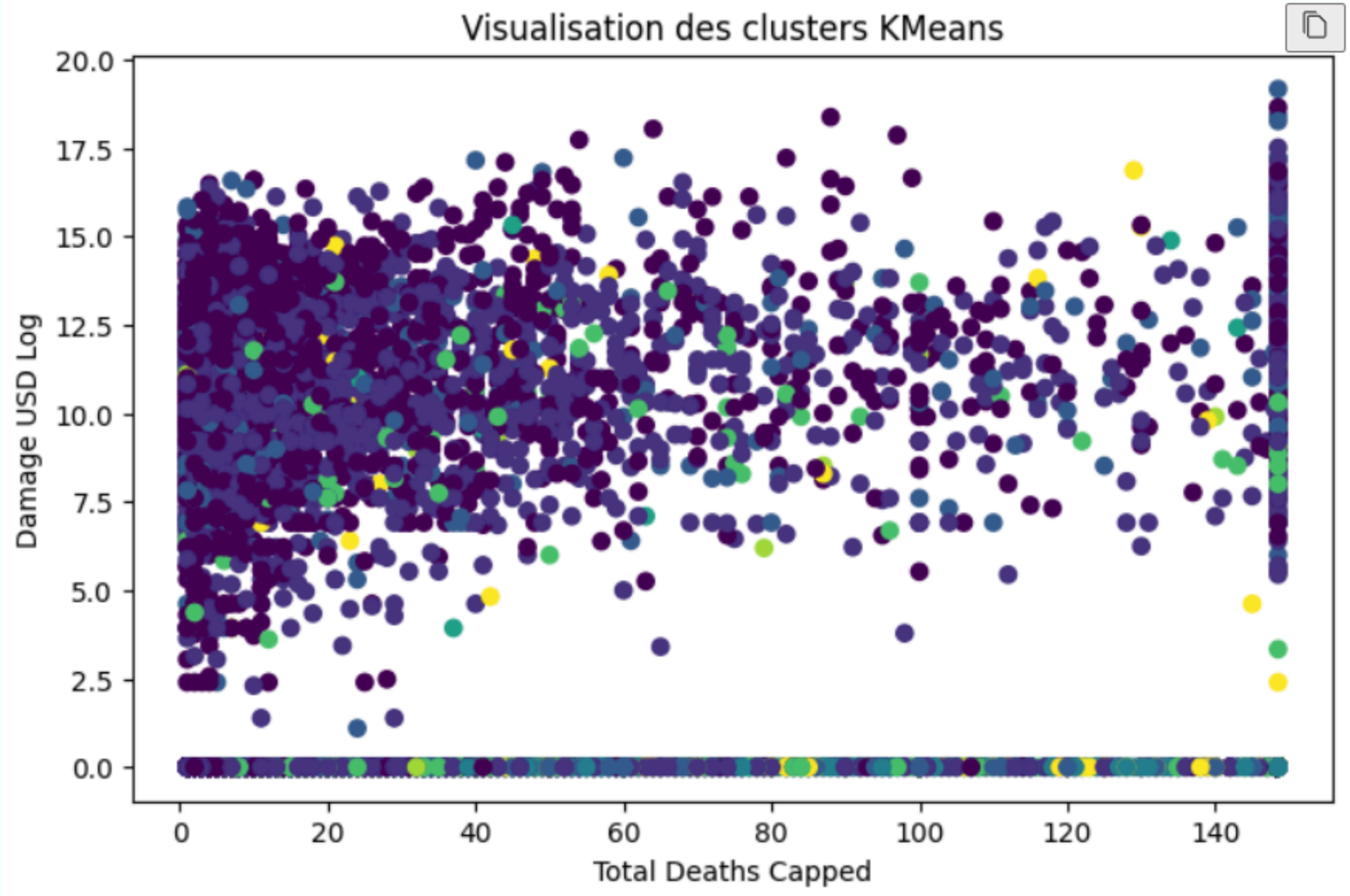

```
--- Évaluation finale du Clustering (KMeans) ---
Nombre de clusters : 8
Inertie : 77224.89324535045
Silhouette Score : 0.4824698020823668

Nombre d'éléments par cluster :
Cluster
0    3512
1    4080
2    1164
3    1266
4      69
5     741
6      91
7     490
Name: count, dtype: int64
```

```
Meilleur nombre de clusters selon Silhouette Score : 8
```

5. Visualisation des clusters KMeans :

Les clusters selon le nombre de morts (après capping) et les dégâts en log où :

- Total_Deaths_Capped représente le nombre de morts après limitation des valeurs extrêmes.
- Damage_Log est une transformation des dégâts économiques pour réduire l'effet des grandes valeurs



IV- Mise en place de la classification

La classification est une méthode supervisée dont l'objectif est de prédire la variable Severe.
Dans cette version, nous utilisons plusieurs splits, ce qui signifie que nous répétons l'entraînement et l'évaluation avec différentes valeurs de random_state.
Cette approche permet d'obtenir une évaluation plus fiable que l'utilisation d'un seul split.

Modèles utilisés:

- **Decision Tree**
- **Logistic Regression**
- **Random Forest**
- **Gradient Boosting**

Méthodologie utilisée :

- **Sélection des variables**

  Nous utilisons les variables suivantes :
  - Damage_Log
  - Year
  - Decade
  - les variables liées au type de catastrophe

```
Nombre de variables pour la classification : 14

Distribution de la variable cible :
Severe
0    9342
1    2071
Name: count, dtype: int64
```

- **Utilisation de plusieurs splits**

Nous utilisons trois splits différents pour améliorer la robustesse des résultats. :

- Split 1 : random_state = 42
- Split 2 : random_state = 7
- Split 3 : random_state = 99

```
seeds = [42, 7, 99]
all_results = []
```

```
# Split Train / Validation / Test
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, random_state=seed, stratify=y )
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, random_state=seed, stratify=y_temp )
print("Train size :", X_train.shape)
print("Validation size :", X_val.shape)
print("Test size :", X_test.shape)
```

```
==============================
Validation avec random_state = 42
==============================
Train size : (7989, 14)
Validation size : (1712, 14)

==============================
Validation avec random_state = 7
==============================
Train size : (7989, 14)
Validation size : (1712, 14)

==============================
Validation avec random_state = 99
==============================
Train size : (7989, 14)
Validation size : (1712, 14)
```

--> Pour chaque split, les mêmes modèles sont entraînés et évalués. Ensuite, nous calculons la moyenne des résultats pour chaque modèle.

V- Évaluation des modèles

- **Évaluation avec plusieurs splits:**
  - Comparaison des performnces sur plusieurs splits
  - Utilisation des métriques : Accuracy, Precision, Recall et F1-score
  - Sélection du meilleur modèle basée sur la moyenne du F1-score

	Split	Model	Validation Accuracy	Validation Precision	Validation Recall	Validation F1-Score
0	42	Logistic Regression	0.695678	0.337461	0.700965	0.455590
1	42	Decision Tree	0.723131	0.332649	0.520900	0.406015
2	42	Random Forest	0.759930	0.369110	0.453376	0.406926
3	42	Gradient Boosting avec Early Stopping	0.827103	0.560000	0.225080	0.321101
4	7	Logistic Regression	0.688084	0.326625	0.680645	0.441423
5	7	Decision Tree	0.723715	0.331959	0.519355	0.405031
6	7	Random Forest	0.752336	0.356784	0.458065	0.401130
7	7	Gradient Boosting avec Early Stopping	0.833528	0.612613	0.219355	0.323040
8	99	Logistic Regression	0.694509	0.335394	0.700000	0.453501
9	99	Decision Tree	0.715537	0.324056	0.525806	0.400984
10	99	Random Forest	0.745327	0.347826	0.464516	0.397790
11	99	Gradient Boosting avec Early Stopping	0.828271	0.593023	0.164516	0.257576

- **Moyenne des performances :**
  - Comparaison des performnces sur plusieurs splits
  - Utilisation des métriques : Accuracy, Precision, Recall et F1-score
  - Sélection du meilleur modèle basée sur la moyenne du F1-score

Dans cette étape on va :

- Calculer la moyenne et l'écart-type des performances pour chaque modèle.

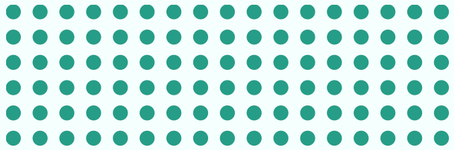
--> Le meilleur modèle est sélectionné en fonction de la moyenne du F1-score

	Model	Validation Accuracy Mean	Validation Accuracy Std	Test Accuracy Mean	Test Accuracy Std	Precision Mean	Precision Std	Recall Mean	Recall Std	F1-Score Mean	F1-Score Std
2	Logistic Regression	0.712553	0.009511	0.715034	0.008734	0.275870	0.005414	0.748124	0.020921	0.403025	0.006027
0	Decision Tree	0.765054	0.003841	0.769326	0.002711	0.283366	0.009108	0.520900	0.047366	0.366849	0.019429
3	Random Forest	0.794819	0.003047	0.791374	0.002814	0.298895	0.009294	0.464094	0.043892	0.363361	0.019795
1	Gradient Boosting avec Early Stopping	0.877635	0.002148	0.876120	0.002387	0.593190	0.030675	0.112540	0.025113	0.188708	0.036672

- **Sélection du meilleur modèle :**

→ Le meilleur modèle est celui qui obtient la meilleure moyenne du F1-score sur l'ensemble des splits

```
# Étape 21 : Sélection du meilleur modèle

best_model_name = summary_results.iloc[0]["Model"]

print("Le meilleur modèle est :", best_model_name)
print("\nPerformances moyennes du meilleur modèle :")
print(summary_results.iloc[0])
```



```
Le meilleur modèle est : Logistic Regression

Performances moyennes du meilleur modèle :
Model                        Logistic Regression
Validation Accuracy Mean                0.712553
Validation Accuracy Std                 0.009511
Test Accuracy Mean                      0.715034
Test Accuracy Std                       0.008734
Precision Mean                           0.27587
Precision Std                           0.005414
Recall Mean                             0.748124
Recall Std                              0.020921
F1-Score Mean                           0.403025
F1-Score Std                            0.006027
Name: 2, dtype: object
```

→ **Le modèle Logistic Regression présente le meilleur équilibre global entre précision et rappel.**

→ **Le modèle favorise le rappel, ce qui est important dans notre cas car il est préférable de détecter un maximum de catastrophes sévères.**

- **Visualisation de la matrice de confusion :**

La matrice de confusion permet d’analyser les bonnes prédictions ainsi que les erreurs du meilleur modèle

Elle permet de :

→ Voir combien de prédictions sont correctes:

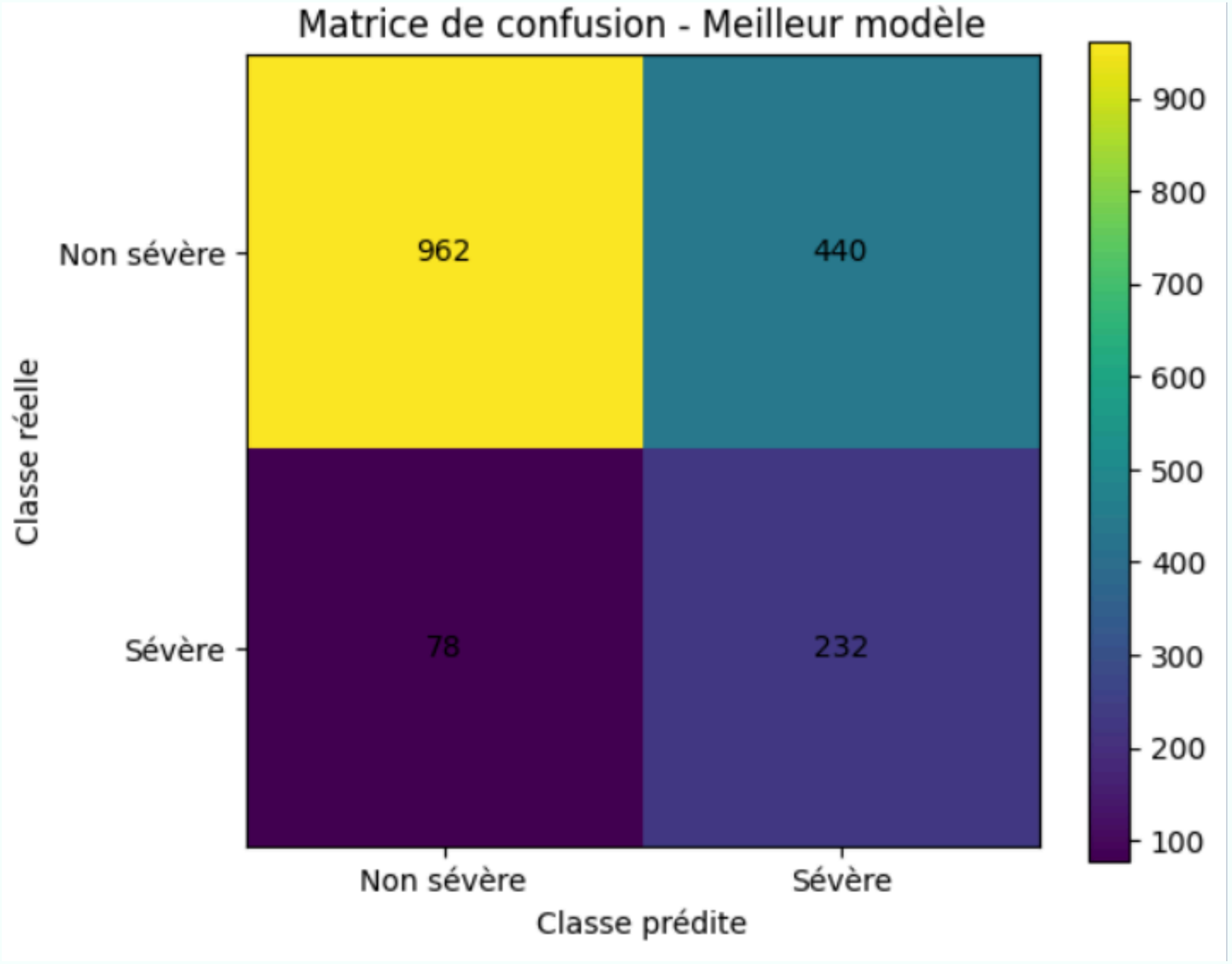
1526 : non sévère correctement prédit

226 : sévère correctement prédit

→ Identifier les erreurs du modèle :

582 : non sévère prédit à tort comme sévère

85 : sévère prédit à tort comme non sévère



Conclusion

Ce projet nous a permis d’appliquer les techniques de Data Mining à l’analyse des catastrophes naturelles, en utilisant K-Means et des modèles de classification pour évaluer leur gravité. À l’avenir, l’intégration de données supplémentaires et de modèles plus avancés pourrait améliorer l’analyse des risques et contribuer à une meilleure prévention.

MERCI !

We hope this presentation was insightful